



UANL

UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN



FIME

FACULTAD DE INGENIERÍA MECÁNICA Y ELÉCTRICA

Universidad Autónoma de Nuevo León

Facultad de Ingeniería Mecánica y Eléctrica

Lenguajes de Programación y Lab

Actividad Fundamental 5: Reporte de Lenguajes Script

Unidad 2

Semestre: 3ro

Equipo: 04

Grupo: 025

Salón: 3109

Periodo: Enero – Junio

Docente: Ing. Selene Guadalupe Pinal Torres

Hora: M4-M6

Nombre	Matricula	Carrera
Ruedas Vazquez Jordan Emanuel	2103677	ITS
Enríquez Nungaray Héctor Miguel	2042781	IAS
Hernández Silva Carlos Yahir	1975208	ITS
Heredia López Betsy Aracely	2054413	IAS

Viernes 28 de Marzo del 2025

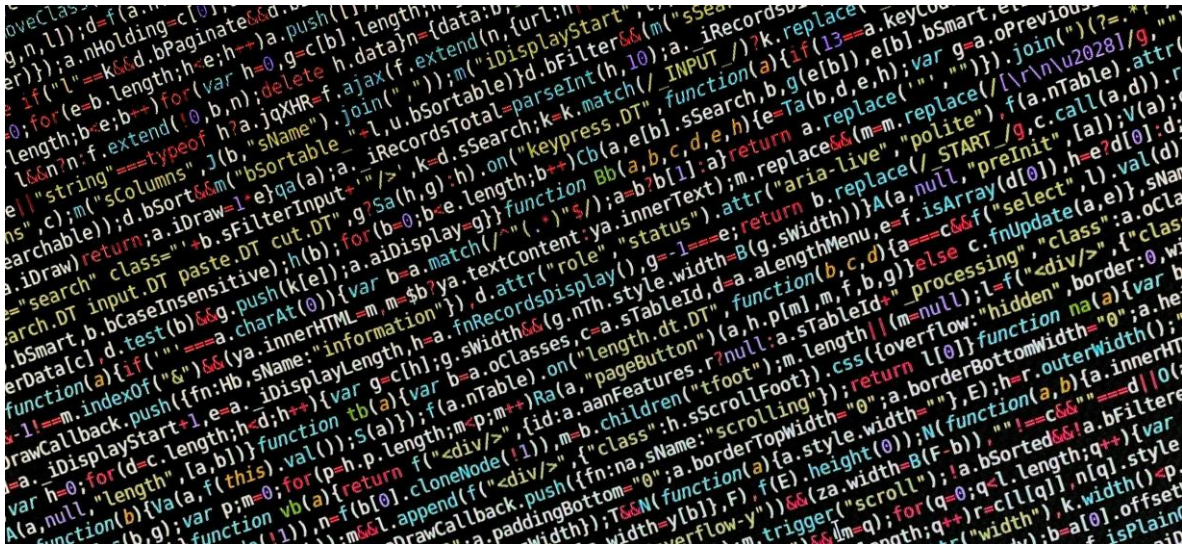
Monterrey, Nuevo León

Introducción

Los lenguajes de script son lenguajes de programación diseñados para ejecutarse en tiempo real mediante un intérprete, sin necesidad de compilación previa. Su principal función es automatizar tareas específicas, como la manipulación de datos, la gestión de servidores o la creación de contenido dinámico en aplicaciones web. A diferencia de los lenguajes compilados, como C++ o Java, estos lenguajes destacan por su sintaxis sencilla y flexible, lo que facilita su aprendizaje y uso en escenarios que requieren rapidez y adaptabilidad.

Su relevancia abarca múltiples ámbitos tecnológicos. En el desarrollo web, por ejemplo, JavaScript permite agregar interactividad a las páginas, mientras que PHP y Python se emplean en el backend para gestionar bases de datos y lógica de servidor. En la administración de sistemas, herramientas como Bash (en Linux) o PowerShell (en Windows) automatizan tareas repetitivas, como copias de seguridad o configuraciones de red. Además, su capacidad para integrarse con otros lenguajes y plataformas los convierte en aliados indispensables en proyectos de análisis de datos, inteligencia artificial y pruebas de software.

La combinación de portabilidad, eficiencia y una amplia comunidad de soporte ha consolidado a los lenguajes de script como pilares en la industria tecnológica. Su uso no solo optimiza procesos, sino que también democratiza el acceso a la programación, permitiendo que tanto principiantes como expertos resuelvan problemas complejos con soluciones elegantes y escalables.



Definición y Características de los Lenguajes Script

Los lenguajes de script son un tipo de lenguaje de programación diseñado para ejecutarse de manera interpretada, es decir, sin necesidad de una compilación previa. A diferencia de los lenguajes compilados, que requieren convertir el código fuente en un archivo ejecutable antes de su ejecución, los lenguajes de script son procesados línea por línea por un intérprete en tiempo real. Esta característica los hace ideales para tareas que demandan flexibilidad y rapidez en la implementación, como la automatización de procesos, la administración de sistemas o el desarrollo de aplicaciones web dinámicas. Su naturaleza interpretada facilita la depuración y las pruebas iterativas, ya que los cambios en el código pueden aplicarse y verificarse inmediatamente.

Diferencias entre Lenguajes Compilados e Interpretados

Los lenguajes compilados, como C, C++ o Rust, requieren que todo el código fuente sea traducido a código máquina antes de ejecutarse. Este proceso genera un archivo binario independiente que puede funcionar sin el código original, lo que garantiza un alto rendimiento en tareas intensivas. Sin embargo, esta ventaja se ve contrarrestada por una menor flexibilidad, ya que cualquier modificación en el código obliga a recompilar el programa completo.

Por otro lado, los lenguajes interpretados, como Python, JavaScript o Bash, no necesitan este paso previo. En su lugar, un intérprete ejecuta las instrucciones directamente línea por línea. Aunque esto puede resultar en un menor rendimiento en comparación con los lenguajes compilados, ofrece ventajas significativas en términos de portabilidad, facilidad de uso y adaptabilidad. Por ejemplo, un script en Python puede modificarse y ejecutarse en múltiples sistemas operativos sin cambios, siempre que exista un intérprete instalado.

Rasgos Clave de los Lenguajes Script

Ejecución sin compilación

La ausencia de compilación permite un desarrollo ágil y dinámico. Los programadores pueden escribir, probar y ajustar el código en tiempo real, lo que acelera la creación de prototipos y la resolución de errores. Este enfoque es especialmente útil en entornos donde los requisitos cambian con frecuencia, como en el desarrollo web o la automatización de tareas administrativas.

Sintaxis sencilla y flexible

Estos lenguajes priorizan la legibilidad y la simplicidad. Por ejemplo, Python utiliza indentación para definir bloques de código, eliminando la necesidad de llaves o puntos y comas. JavaScript, por su parte, permite

manejar eventos en navegadores con pocas líneas de código. Esta claridad sintáctica reduce la curva de aprendizaje y facilita la colaboración entre equipos.

Usos principales

- **Automatización:** Herramientas como Bash o PowerShell permiten crear scripts para gestionar archivos, programar copias de seguridad o configurar servidores de forma automática.
- **Servidores:** Lenguajes como PHP o Python (con frameworks como Django o Flask) son esenciales en el backend de aplicaciones web, donde manejan lógica de negocio, conexiones a bases de datos y APIs.
- **Desarrollo web:** JavaScript domina el frontend, añadiendo interactividad a páginas web, mientras que Node.js extiende sus capacidades al lado del servidor.

Tipos de Lenguajes Script

Los lenguajes de script se clasifican en diversas categorías según su enfoque y ámbito de aplicación. A continuación, se detallan los principales tipos, sus características, ejemplos prácticos y casos de uso.

Lenguajes para Web

JavaScript/ECMAScript

JavaScript, basado en el estándar ECMAScript, es un lenguaje de scripting multipropósito ampliamente utilizado en desarrollo web. Sus funciones de primera clase permiten tratar las funciones como variables, mientras que su enfoque en programación orientada a objetos basada en prototipos facilita la reutilización de código. Un ejemplo ilustrativo es un bucle que suma números del 1 al 10:

```
```\javascript
let total = 0, count = 1;
while (count <= 10) {
 total += count;
 count += 1;
}
console.log(total); // Resultado: 55
```
```

JavaScript se ejecuta en navegadores mediante motores como V8 (Chrome) y SpiderMonkey (Mozilla). Además, con Node.js, se extiende al backend, permitiendo crear servidores y APIs. También integra bases de datos NoSQL como MongoDB, lo que lo convierte en un pilar del desarrollo full-stack.

PHP

PHP, originalmente diseñado para páginas web dinámicas, destaca por su sintaxis similar a C y su capacidad para incrustarse en HTML. Es código abierto y soporta programación orientada a objetos. Un script típico en PHP podría duplicar los valores de un arreglo:

```
```\php
<?php
$arr = array(1, 2, 3, 4);
foreach ($arr as &$value) {
```

```
$value = $value * 2;
}
unset($value);
?>
````
```

Se utiliza en servidores como Apache y Nginx, y es la base de CMS como WordPress y frameworks como Laravel. Su flexibilidad lo mantiene relevante en aplicaciones web empresariales.

Python

Python, reconocido por su sintaxis clara y legibilidad, soporta múltiples paradigmas, incluyendo programación estructurada y orientada a objetos. Su extensa biblioteca estándar simplifica tareas comunes. Un ejemplo básico es un bucle que imprime números del 0 al 4:

```
```python
count = 0
while count < 5:
 print(count)
 count += 1
```
```

Se emplea en desarrollo web con frameworks como Django, aprendizaje automático, automatización de procesos y análisis de datos. Su naturaleza multiplataforma permite ejecutarlo en Linux, Windows y macOS.

Shell Scripting

Bash (Linux)

Bash, predominante en sistemas Unix/Linux, ofrece una sintaxis sencilla ideal para automatizar tareas administrativas. Por ejemplo, un script que suma números del 1 al 10:

```
```bash
#!/bin/bash
total=0
```

```

count=1
while [$count -le 10]; do
 total=$((total + count))
 count=$((count + 1))
done
echo "El total es: $total"
'''

```

Se usa en administración de servidores, copias de seguridad y automatización de procesos, siendo esencial para administradores de sistemas.

### ***PowerShell***

PowerShell, desarrollado por Microsoft, destaca por su integración con objetos .NET y su capacidad multiplataforma. Un comando como `'Copy-Item -Path C:\boot.ini -Destination C:\boot.bak'` demuestra su utilidad en la gestión de archivos. Es clave en entornos Windows para automatizar configuraciones y administrar servicios, aunque también funciona en Linux y macOS.

## **Scripting en Automatización**

### ***Ruby***

Ruby, con su sintaxis elegante y enfoque en orientación a objetos, permite crear aplicaciones robustas con poco código. Un ejemplo es una clase que hereda de `'Song'` para manejar letras de karaoke:

```

```ruby
class KaraokeSong < Song
  def initialize(name, artist, duration, lyrics)
    super(name, artist, duration)
    @lyrics = lyrics
  end
end
'''

```

Es popular en desarrollo web gracias a Ruby on Rails, utilizado por plataformas como GitHub y Shopify, y en herramientas como Sass.

Perl

Perl, conocido por su potencia en procesamiento de texto, se usa en scripts CGI y manipulación de datos. Un subprograma que calcula el cuadrado de un número ilustra su sintaxis:

```
```perl
sub square {
 my $num = shift;
 my $result = $num * $num;
 return $result;
}
$sq = square(8);
```
```

Aunque menos popular hoy, sigue siendo relevante en backend, administración de sistemas y sitios como Amazon.

Ejemplo Práctico en Python

Un sistema de gestión de recursos humanos en Python demuestra la versatilidad de los lenguajes de script. El código define una clase 'Empleado' con métodos para calcular edades y generar resúmenes, y una clase 'GestorRecursosHumanos' que administra empleados y exporta reportes a CSV:

```
```python
class Empleado:
 def __init__(self, id, nombre, apellido, fecha_nacimiento, salario, departamento):
 self.id = id
 self.nombre = nombre
 self.apellido = apellido
 self.fecha_nacimiento = fecha_nacimiento
 self.salario = salario
 self.departamento = departamento

 def calcular_edad(self):
 hoy = date.today()
 edad = hoy.year - self.fecha_nacimiento.year
 if (hoy.month, hoy.day) < (self.fecha_nacimiento.month, self.fecha_nacimiento.day):
```



```

 edad -= 1
 return edad

def generar_resumen(self):
 return {
 "ID": self.id,
 "Nombre Completo": f'{self.nombre} {self.apellido}',
 "Edad": self.calcular_edad(),
 "Departamento": self.departamento,
 "Salario": f'{self.salario:.2f}'
 }
'''

```

Este script ilustra cómo Python maneja datos complejos, integra bibliotecas como `csv` y `datetime`, y facilita la creación de soluciones empresariales escalables. Algunas tecnologías modernas combinan ambos enfoques mediante **compilación en tiempo de ejecución (Just-In-Time, JIT)**, donde el código fuente se traduce parcialmente antes de la ejecución y se optimiza mientras el programa está en uso. Java y C# son ejemplos de lenguajes que utilizan este método, lo que les permite ofrecer un buen equilibrio entre rendimiento y flexibilidad. La elección entre un compilador o un intérprete depende de factores como la necesidad de rendimiento, la facilidad de depuración y la naturaleza del software desarrollado. En general, los compiladores son preferidos para aplicaciones que requieren alta velocidad de ejecución, mientras que los intérpretes son ideales para entornos de desarrollo dinámicos y pruebas rápidas.

## **Ventajas y Desventajas de los Lenguajes de Script**

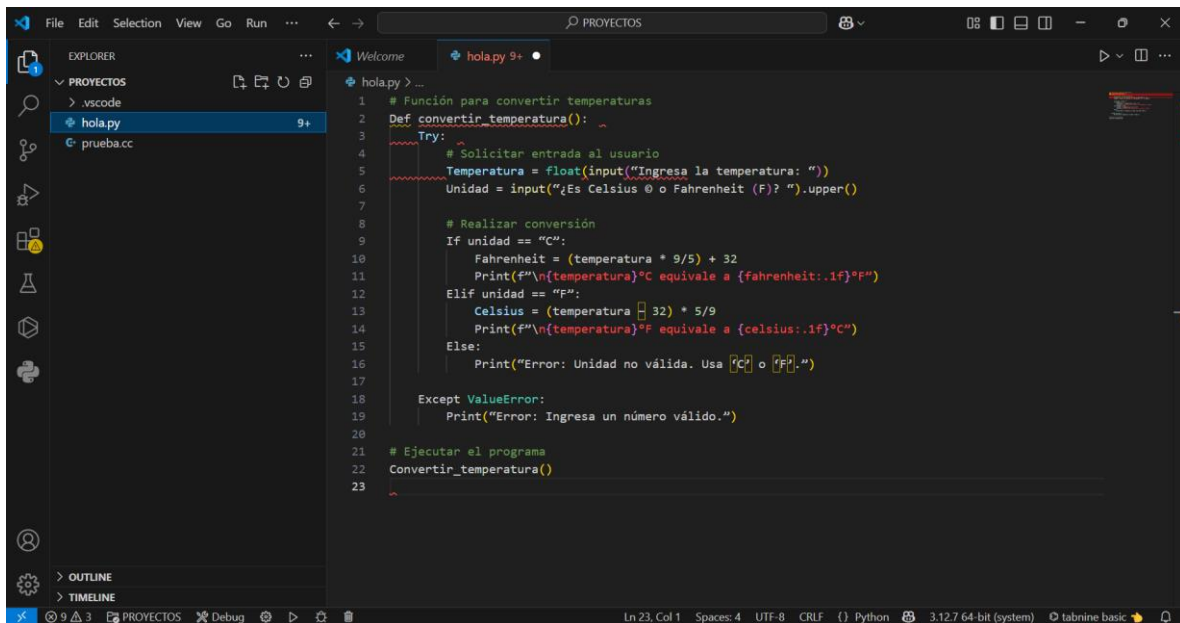
Los lenguajes de script destacan por su agilidad en el desarrollo, una cualidad derivada de su naturaleza interpretada. Al prescindir de la compilación, los programadores pueden escribir, probar y modificar código en tiempo real, lo que acelera la creación de prototipos y la corrección de errores. Esta rapidez es especialmente valiosa en entornos dinámicos como el desarrollo web, donde los requisitos cambian con frecuencia, o en la automatización de tareas administrativas, donde un script en Bash o PowerShell puede simplificar procesos repetitivos en minutos. Además, su sintaxis simplificada, como la indentación en Python o la flexibilidad de JavaScript, reduce la curva de aprendizaje y fomenta la colaboración entre equipos, incluso en proyectos multidisciplinarios. La portabilidad es otra ventaja clave: un script escrito en Python puede ejecutarse en Windows, Linux o macOS sin modificaciones, siempre que exista un intérprete instalado, lo que facilita la integración en entornos heterogéneos.

Sin embargo, esta flexibilidad conlleva limitaciones en rendimiento, particularmente en aplicaciones que demandan alto procesamiento. Los lenguajes interpretados, al ejecutarse línea por línea, suelen ser más lentos que los compilados como C++ o Java, lo que los hace menos adecuados para software de cálculo intensivo, motores gráficos o sistemas embebidos. Otro desafío es la dependencia del intérprete, ya que la ausencia de este en un sistema impedirá la ejecución del script, limitando su autonomía en entornos restringidos o legacy. Además, la libertad sintáctica, aunque útil, puede derivar en malas prácticas si no se aplican estándares de codificación, generando código difícil de mantener o escalar en proyectos complejos.

A pesar de estos inconvenientes, los lenguajes de script mantienen su relevancia al equilibrar eficiencia y versatilidad. En ámbitos como el desarrollo web frontend, donde JavaScript domina la interactividad, o en la automatización de infraestructuras con herramientas como Ansible (basado en Python), sus ventajas superan ampliamente las desventajas. Incluso en campos emergentes como la inteligencia artificial, Python se erige como un estándar gracias a bibliotecas como TensorFlow y a su capacidad para integrarse con lenguajes de bajo nivel. Así, mientras su rendimiento puede no ser óptimo para todas las aplicaciones, su adaptabilidad, soporte comunitario y enfoque en la productividad los consolidan como pilares en la era de la automatización y el desarrollo ágil.

## Ejemplos:

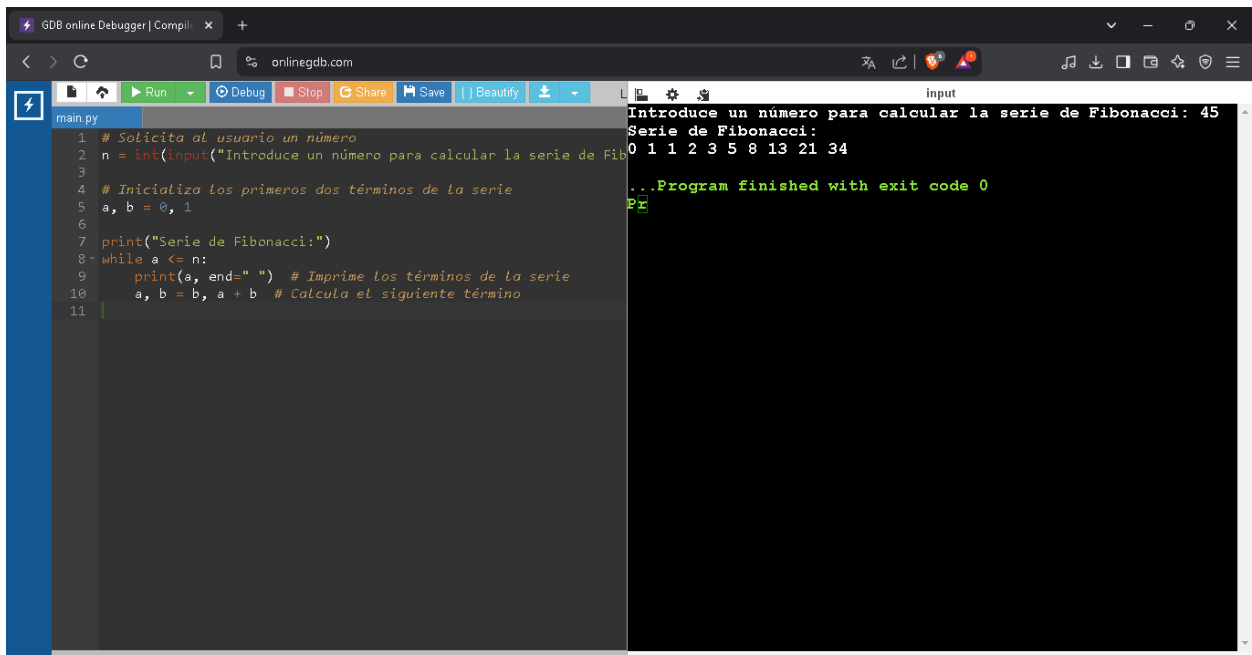
- Ruedas Vazquez Jordan Emanuel



The screenshot shows the Visual Studio Code editor with a Python file named `hola.py`. The code is a function `convertir_temperatura()` that takes user input for temperature and unit, then converts it. The code is as follows:

```
1 # Función para convertir temperaturas
2 def convertir_temperatura():
3 try:
4 # Solicitar entrada al usuario
5 Temperatura = float(input("Ingresa la temperatura: "))
6 Unidad = input("¿Es Celsius o Fahrenheit (F)? ").upper()
7
8 # Realizar conversión
9 if Unidad == "C":
10 Fahrenheit = (Temperatura * 9/5) + 32
11 Print(f"\n{Temperatura}°C equivale a {Fahrenheit:.1f}°F")
12 elif Unidad == "F":
13 Celsius = (Temperatura - 32) * 5/9
14 Print(f"\n{Temperatura}°F equivale a {Celsius:.1f}°C")
15 else:
16 Print("Error: Unidad no válida. Usa [C] o [F].")
17
18 except ValueError:
19 Print("Error: Ingresa un número válido.")
20
21 # Ejecutar el programa
22 convertir_temperatura()
23
```

- Enríquez Nungaray Héctor Miguel



The screenshot shows the GDB online Debugger interface with a Python file named `main.py`. The code is a script that calculates the Fibonacci series for a given number `n`. The code is as follows:

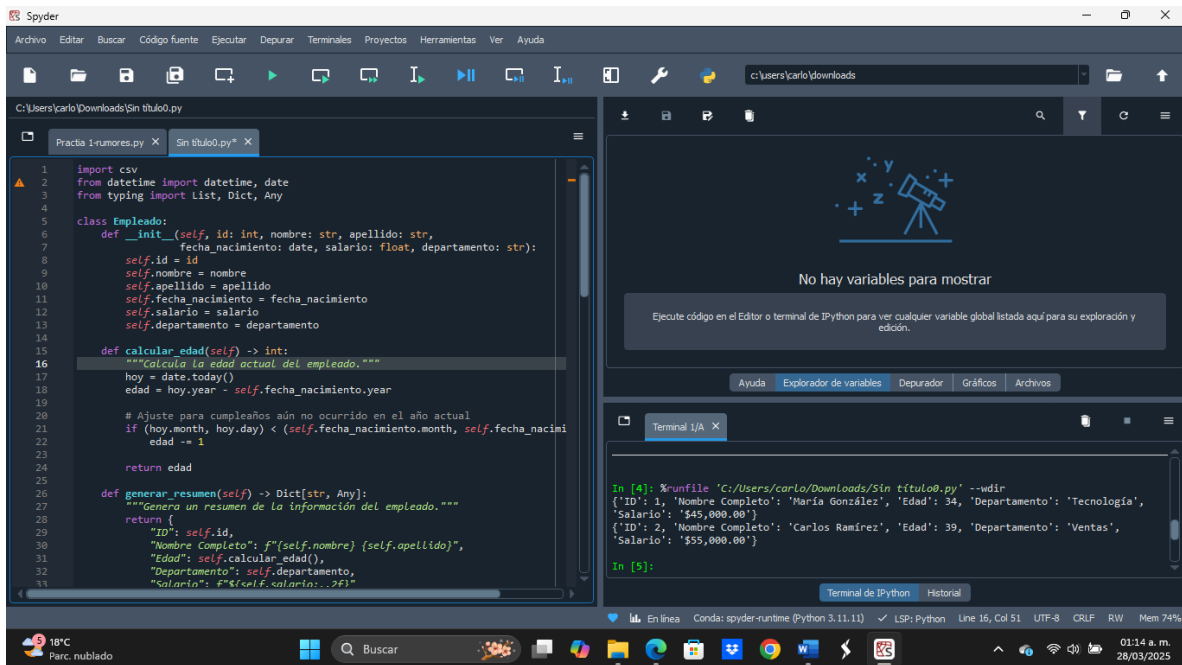
```
1 # Solicita al usuario un número
2 n = int(input("Introduce un número para calcular la serie de Fibonacci: "))
3
4 # Inicializa los primeros dos términos de la serie
5 a, b = 0, 1
6
7 print("Serie de Fibonacci:")
8 while a <= n:
9 print(a, end=" ") # Imprime los términos de la serie
10 a, b = b, a + b # Calcula el siguiente término
11
```

The output of the program is shown in the terminal window:

```
Introduce un número para calcular la serie de Fibonacci: 45
Serie de Fibonacci:
0 1 1 2 3 5 8 13 21 34
...Program finished with exit code 0
```

Explicación:

- Se solicita al usuario un número límite para calcular la serie.
  - Se inicializan los dos primeros números de Fibonacci (0 y 1).
  - Se usa un bucle while para generar la serie hasta el número indicado.
- 
- Hernández Silva Carlos Yahí



Este script demuestra un sistema de gestión de recursos humanos, utilizando programación orientada a objetos, manejo de tipos, cálculos de fecha, y exportación de datos. Ilustra la potencia y flexibilidad de los lenguajes script en el procesamiento de información empresarial.

Import csv

From datetime import datetime, date

From typing import List, Dict, Any

Class Empleado:

Def \_\_init\_\_(self, id: int, nombre: str, apellido: str,

```

 Fecha_nacimiento: date, salario: float, departamento: str):

 Self.id = id

 Self.nombre = nombre

 Self.apellido = apellido

 Self.fecha_nacimiento = fecha_nacimiento

 Self.salario = salario

 Self.departamento = departamento

Def calcular_edad(self) -> int:

 """Calcula la edad actual del empleado."""

 Hoy = date.today()

 Edad = hoy.year - self.fecha_nacimiento.year

 # Ajuste para cumpleaños aún no ocurrido en el año actual

 If (hoy.month, hoy.day) < (self.fecha_nacimiento.month, self.fecha_nacimiento.day):

 Edad -= 1

 Return edad

Def generar_resumen(self) -> Dict[str, Any]:

 """Genera un resumen de la información del empleado."""

 Return {

 "ID": self.id,

 "Nombre Completo": f"{self.nombre} {self.apellido}",

 "Edad": self.calcular_edad(),

 "Departamento": self.departamento,

```

```
 "Salario": f"${self.salario:,.2f}"
 }
}
```

Class GestorRecursosHumanos:

```
Def __init__(self):
```

```
 Self.empleados: List[Empleado] = []
```

```
Def agregar_empleado(self, empleado: Empleado):
```

```
 """Agrega un nuevo empleado al sistema."""
```

```
 Self.empleados.append(empleado)
```

```
Def buscar_empleado_por_id(self, id: int) -> Empleado:
```

```
 """Busca un empleado por su ID."""
```

```
 For empleado in self.empleados:
```

```
 If empleado.id == id:
```

```
 Return empleado
```

```
 Raise ValueError(f"No se encontró empleado con ID {id}")
```

```
Def generar_reporte_plantilla(self) -> List[Dict[str, Any]]:
```

```
 """Genera un reporte de todos los empleados."""
```

```
 Return [empleado.generar_resumen() for empleado in self.empleados]
```

```
Def exportar_reporte_csv(self, nombre_archivo: str):
```

```
 """Exporta el reporte de empleados a un archivo CSV."""
```

```
 With open(nombre_archivo, 'w', newline="", encoding='utf-8') as archivo_csv:
```

```
 # Determinar los campos basados en el primer resumen de empleado
```

```

 If self.empleados:

 Campos = list(self.empleados[0].generar_resumen().keys())

 Escritor = csv.DictWriter(archivo_csv, fieldnames=campos)

 # Escribir encabezados

 Escritor.writeheader()

 # Escribir datos de empleados

 For empleado in self.empleados:

 Escritor.writerow(empleado.generar_resumen())

Ejemplo de uso del script

Def main():

 # Crear sistema de gestión de recursos humanos

 Rrhh = GestorRecursosHumanos()

 # Agregar empleados

 Rrhh.agregar_empleado(Empleado(

 Id=1,

 Nombre="María",

 Apellido="González",

 Fecha_nacimiento=date(1990, 5, 15),

 Salario=45000.00,

 Departamento="Tecnología"

))

```

```
Rrh.agregar_empleado(Empleado(
 Id=2,
 Nombre="Carlos",
 Apellido="Ramírez",
 Fecha_nacimiento=date(1985, 11, 22),
 Salario=55000.00,
 Departamento="Ventas"
))

Generar y mostrar reporte

Reporte = rrh.generar_reporte_plantilla()

For entrada in reporte:

 Print(entrada)

Exportar reporte a CSV

Rrh.exportar_reporte_csv("reporte_empleados.csv")

If __name__ == "__main__":

 Main()
```



## Conclusión General

Los lenguajes de scripting han revolucionado la forma en que abordamos desafíos tecnológicos, posicionándose como herramientas fundamentales en campos como el desarrollo de software, la administración de sistemas y el análisis de datos. Su naturaleza interpretada elimina barreras técnicas, permitiendo a los desarrolladores probar y ajustar código de manera ágil, sin los rigores de la compilación. Esta flexibilidad, sumada a una sintaxis clara y accesible, los hace ideales para proyectos que demandan rapidez y adaptabilidad, desde la creación de interfaces web dinámicas hasta la automatización de flujos de trabajo en entornos empresariales.

Más allá de su utilidad práctica, estos lenguajes han impulsado la innovación en áreas emergentes. Por ejemplo, Python se ha convertido en un estándar para la inteligencia artificial y el procesamiento de datos masivos, mientras que JavaScript continúa dominando el desarrollo frontend con frameworks como React o Angular. En el ámbito de la ciberseguridad, lenguajes como Bash y Python son esenciales para realizar pruebas de penetración y auditorías de sistemas.

Desde una perspectiva evolutiva, los lenguajes de scripting representan un puente entre la programación tradicional y las demandas de la era digital. Su simplicidad no solo los hace accesibles para nuevos programadores, sino que también permite a expertos construir soluciones robustas con menos código. En un mundo donde la eficiencia y la escalabilidad son críticas, estos lenguajes seguirán siendo pilares tecnológicos, facilitando la creación de herramientas adaptables y fomentando la colaboración global gracias a comunidades activas y recursos compartidos. En definitiva, son un testimonio de cómo la tecnología puede simplificar lo complejo y potenciar la creatividad humana.



## **Conclusiones Individuales**

### **Ruedas Vazquez Jordan Emanuel**

Los lenguajes de script representan un paradigma fundamental en la evolución de la programación, donde la simplicidad y la eficiencia se alinean para resolver problemas complejos con soluciones elegantes. Su capacidad para automatizar tareas repetitivas, integrar sistemas dispares y acelerar el desarrollo de aplicaciones refleja un principio clave de la tecnología moderna: optimizar recursos sin sacrificar versatilidad. Estos lenguajes han democratizado el acceso a la programación, permitiendo que no solo desarrolladores expertos, sino también profesionales de otras áreas, como científicos de datos o administradores de sistemas, implementen soluciones técnicas sin una curva de aprendizaje prohibitiva. Esta accesibilidad ha impulsado la innovación en sectores tan diversos como la inteligencia artificial, la ciberseguridad y el desarrollo web, demostrando que la potencia no siempre reside en la complejidad, sino en la adaptabilidad y la claridad del código.

### **Enríquez Nungaray Héctor Miguel**

Los lenguajes de script son un testimonio de cómo la simplicidad y la adaptabilidad pueden transformar la tecnología. Su capacidad para automatizar tareas, integrar sistemas y acelerar el desarrollo los convierte en herramientas indispensables en campos como la web, la inteligencia artificial y la administración de sistemas. Sin embargo, su verdadero valor radica en saber cuándo usarlos: mientras optimizan procesos ágiles y colaborativos, también nos recuerdan la importancia de equilibrar sus ventajas con las limitaciones técnicas, eligiendo siempre la herramienta adecuada para cada desafío. En un mundo tecnológico en constante evolución, su legado es la democratización de la innovación.

### **Heredia López Betsy Aracely**

Desde mi perspectiva, los lenguajes de scripting representan una evolución clave en la programación, ya que permiten optimizar y automatizar procesos de manera eficiente. Su simplicidad y versatilidad los hacen accesibles tanto para principiantes como para expertos, impulsando la innovación en múltiples áreas, desde el desarrollo web hasta la inteligencia artificial.

### **Hernández Silva Carlos Yahir**

Los lenguajes de script han destacado mucho para el desarrollo de tecnologías, ya que ofrecen muchas ventajas en su uso. Los lenguajes de script seguirán desarrollándose y desarrollando software estos son más rápidos y de fácil uso a comparación de los lenguajes que no son script.

## Referencias Bibliográficas

*Cairo, O. (2024). Algoritmos y estructuras de datos. Editorial Universitaria.*

*Monus, A. (2015, 14 de febrero). Los 13 principales lenguajes de scripts a los que deberías prestar atención en 2025. Kinsta®. <https://kinsta.com/es/blog/lenguajes-script/>*

*Matthes, E. (2019). Python Crash Course: A Hands-On, Project-Based Introduction to Programming (2.a ed.). No Starch Press.*

*Crockford, D. (2008). JavaScript: The Good Parts. O'Reilly Media.*

*PHP Group. (2023). PHP: Hypertext Preprocessor. <https://www.php.net/manual/es/intro-what-is.php>*

*IEEE Computer Society. (2021). Trends in Scripting Languages: Flexibility vs. Performance. IEEE Xplore. <https://ieeexplore.ieee.org/document/9472331>*

*Mozilla Developer Network. (2024). JavaScript Guide. <https://developer.mozilla.org/es/docs/Web/JavaScript/Guide>*

*TypeScript Handbook. (2024). Basic Types. <https://www.typescriptlang.org/docs/handbook/basic-types.html>*